

Second presentation

Marcel Siwiela

The objective of the project

Reading data from several sensors:

- Temperature and humidity – DHT11
- Light intensity – BH1750
- Pressure – BMP280

Data transfer of measured values in the following order:

NUCLEO-G071RB -> ESP8266 ---(WiFi)---> ESP8266 ->
-> NUCLEO-L476RG -> DISPLAY

Assumptions (parameters)

1. Create simple program which displays something on display on stm32-L4
2. Implement sensors on stm32-G017 in order:
 - a) DHT11
 - b) + BMP280
 - c) + BH1750
3. Implement buzzer with conditions
4. Implement WiFi module
5. Make visualisation and send info through wifi

Design must include:

1. Parallel processing
2. Schematic in KiCad
3. No user inputs

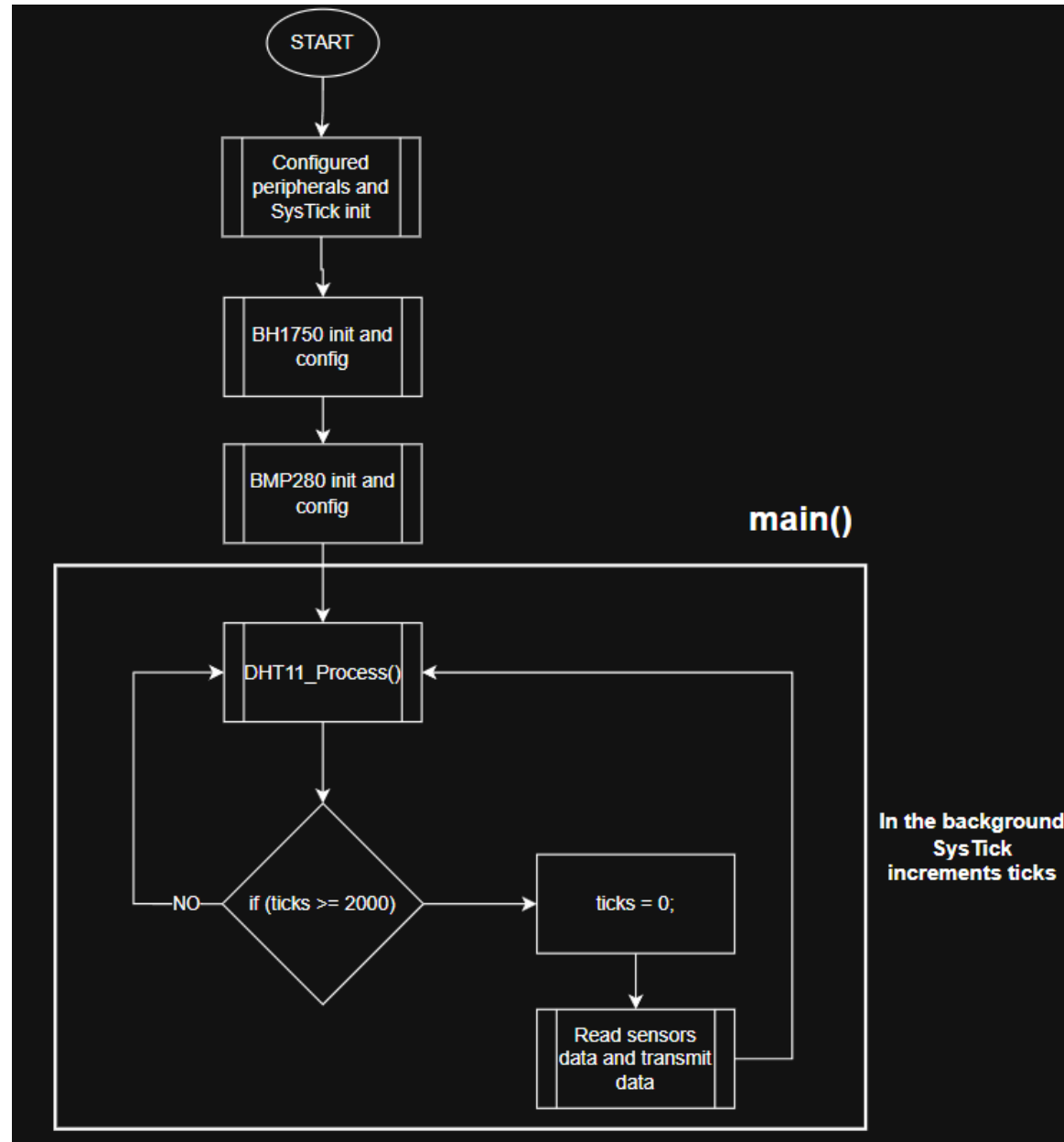
A description of the components and subcomponents used

- NUCLEO-G071RB – sensor based mcu
- ESP8266 x2 (**WiFi**) – for communication via WiFi
- NUCLEO-L476RG – display unit
- Waveshare 4inch TFT Touch Shield – display
- Temperature and humidity – DHT11
- Light intensity x2 – BH1750
- Pressure – BMP280
- Wires, breadboard – for connections
- Buzzer – alarm when humidity $\geq 60\%$

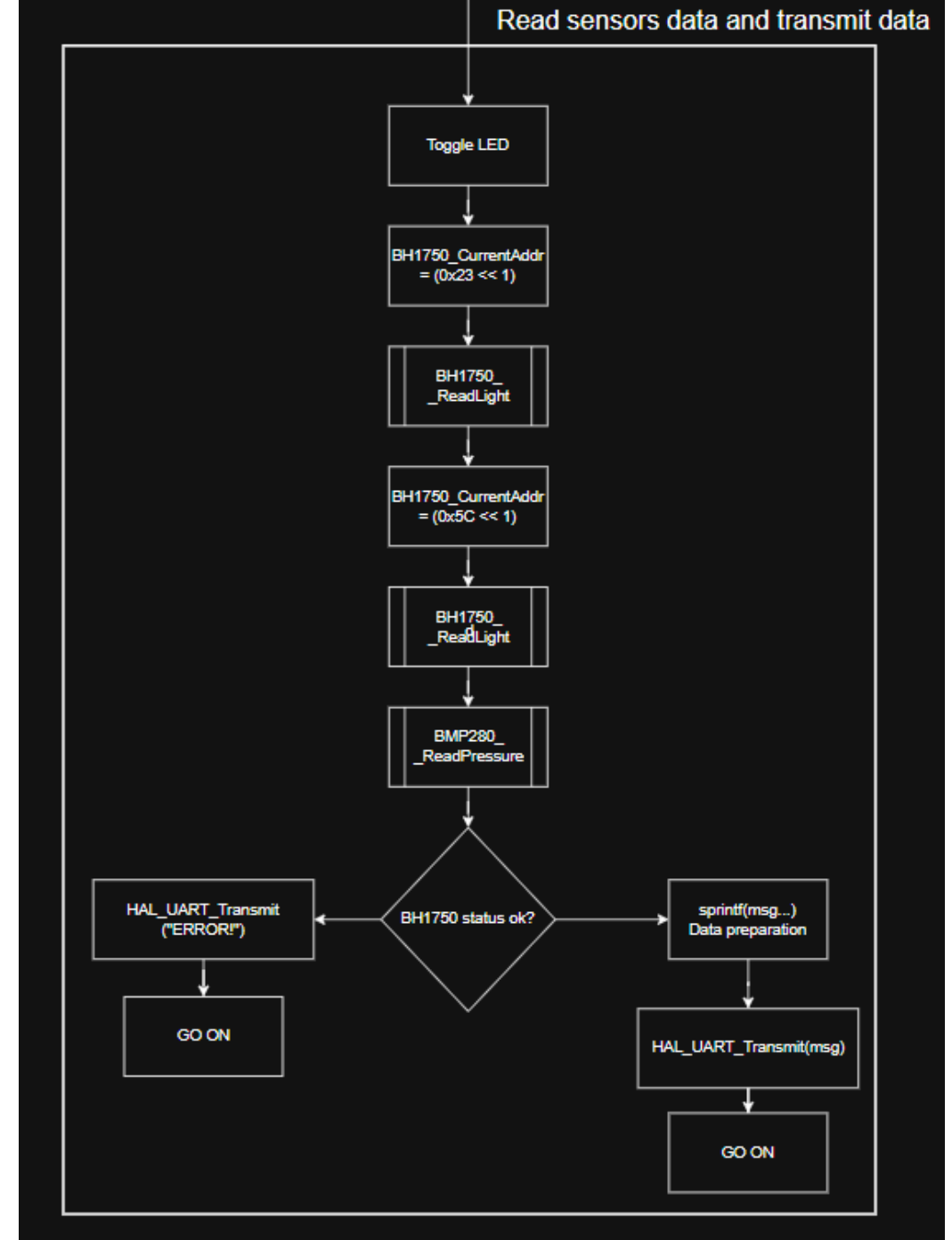
Data Flow Diagram



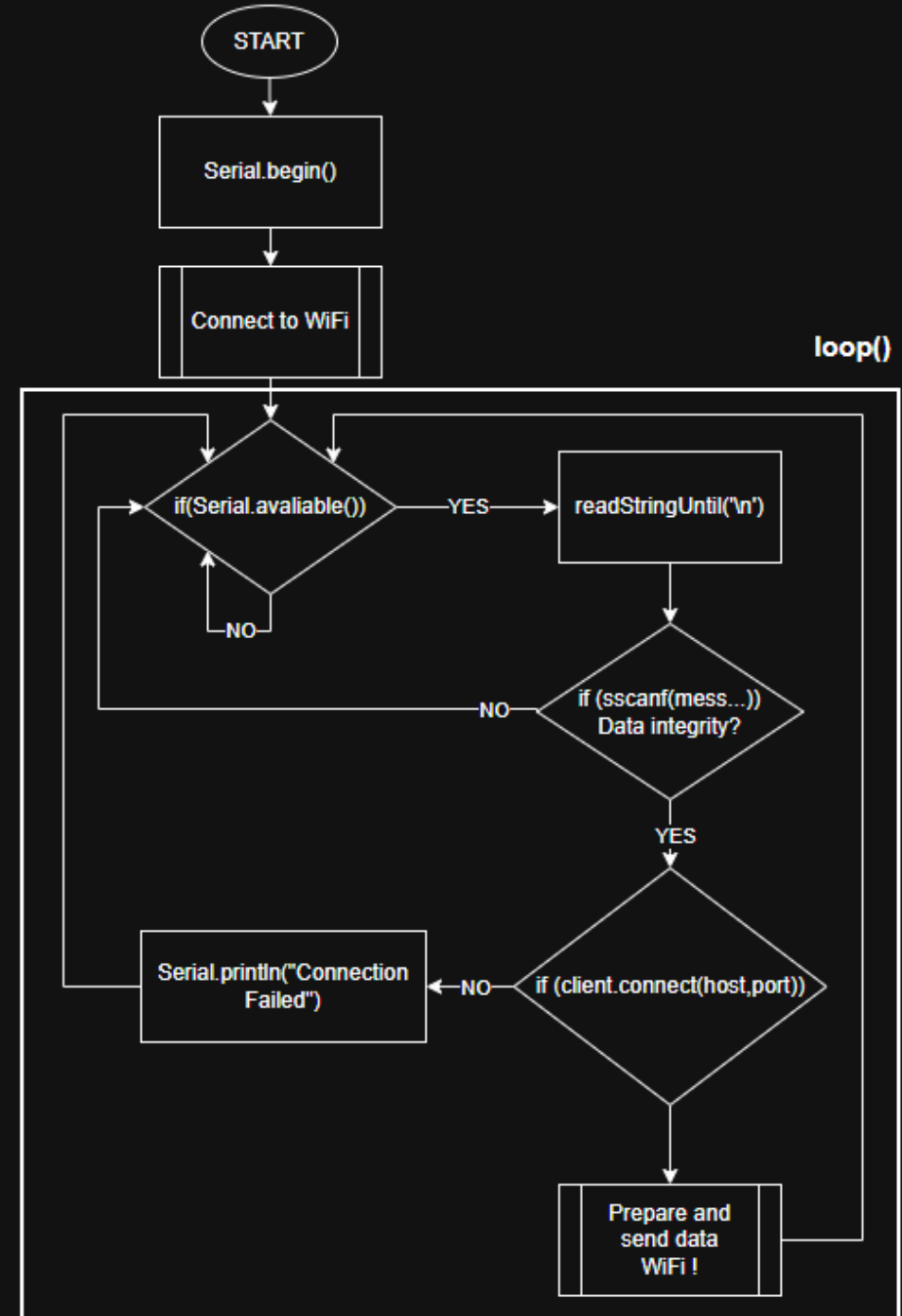
Flowchart – sensor base mcu (G071RB)



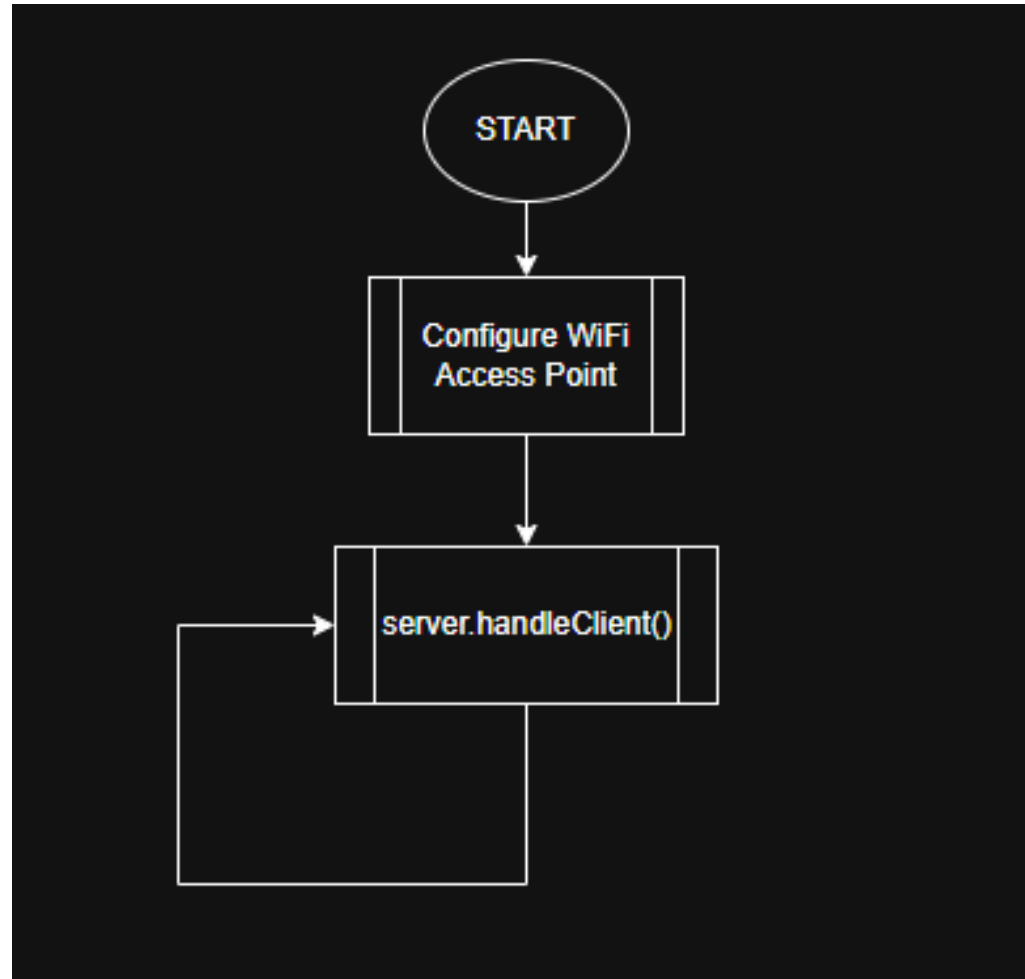
Flowchart – sensor base mcu (G071RB) – read sensors data and transmit data



Flowchart – ESP8266 Tx



Flowchart – ESP8266 Rx

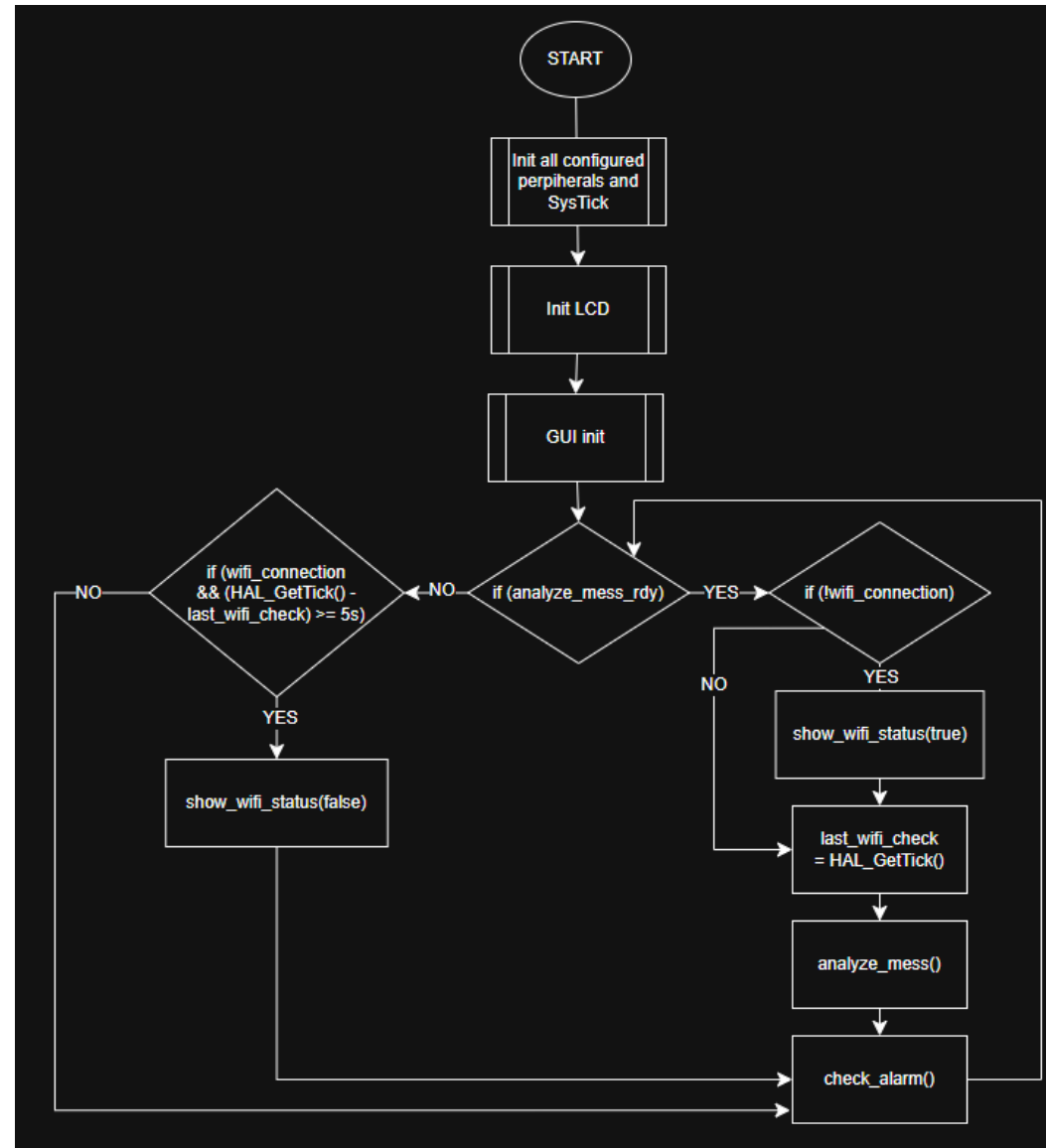


Flowchart – ESP8266 Rx – handleData

```
void handleData()
{
  if (server.hasArg("t")) t = server.arg("t");
  if (server.hasArg("h")) h = server.arg("h");
  if (server.hasArg("p")) p = server.arg("p");
  if (server.hasArg("l_v")) l_v = server.arg("l_v");
  if (server.hasArg("l_g")) l_g = server.arg("l_g");
  String mess = "T" + String(t) + "|H" + String(h) + "|P" + String(p) + "|L_V" + String(l_v) + "|L_G" + String(l_g) + terminator;
  Serial.println(mess);
  mySerial.println(mess);

  server.send(200, "text/plain", "OK");
}
```

Flowchart – NUCLEO-L476RG



Flowchart – NUCLEO-L476RG UART

```
7  USER CODE BEGIN 4  7
⊖ void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == UART4)
    {
        HAL_UART_Transmit(&huart2, &rx_byte, sizeof(rx_byte), 100);
        HAL_UART_Receive_IT(&huart4, &rx_byte, 1);
        analyze_mess_rdy = true;
    }
}
```

Flowchart – NUCLEO-L476RG analyze_mess()

```
void analyze_mess(void)
{
    if (rx_byte == '$')
    {
        rx_mess[j + 1] = '\0';
        // hum external for buzzer
        int temp;
        int l_v, l_g;
        int p;
        int var_num = sscanf((char*)rx_mess, " T%d|H%d|P%d|L_V%d|L_G%d", &temp, &hum, &p, &l_v, &l_g);
        uint32_t vars[] = {temp, hum, p, l_v, l_g};

        if (var_num == 5)
        {
            for (uint8_t i = 1; i < SENSOR_ROWS_LEN; i++)
            {
                update_disp_val(vars[i-1], &sensor_rows[i]);
            }
            j = -1;
        }
        else
        {
            if (j == -1 && (rx_byte == '\r' || rx_byte == '\n')) return;
            if (j < MESS_SIZE - 1)
            {
                rx_mess[++j] = rx_byte;
            }
        }
    }
}
```

Flowchart – NUCLEO-L476RG gui funcs

```
39 void display_text_at_row(const char *text, const DisplayRow *row)
40 {
41     uint16_t x_temp = (row->label_len)*LETTER_WIDTH;
42     GUI_DrawRectangle(x_temp, row->y_pos, SCREEN_WIDTH, row->y_pos + FONT_SIZE, WHITE, DRAW_FULL, DOT_PIXEL_1X1);
43     GUI_DisString_EN(x_temp, row->y_pos, (char*) text, &Font24, WHITE, BLACK);
44 }
45
46 void show_wifi(bool state)
47 {
48     display_text_at_row(state ? "YES" : "NO", &sensor_rows[ROW_WIFI]);
49 }
50
51 void update_disp_val(uint32_t value, const DisplayRow *row)
52 {
53     char buff[16];
54     sprintf(buff, "%" PRIu32 " ", value);
55     display_text_at_row(buff, row);
56 }
57
```

Flowchart – NUCLEO-L476RG alarm

```
8 void check_alarm(void)
9 {
10     if (hum >= 60)
11     {
12         if (HAL_GetTick() - last_tick_time >= 10000)
13         {
14             HAL_GPIO_TogglePin(BUZZER_GPIO_Port, BUZZER_Pin);
15             last_tick_time = HAL_GetTick();
16             buzzer_active = true;
17         }
18     }
19     else
20     {
21         if (buzzer_active)
22         {
23             HAL_GPIO_WritePin(BUZZER_GPIO_Port, BUZZER_Pin, 0);
24             buzzer_active = false;
25         }
26     }
27 }
```

uC peripherals used

- UART – communication between stm and esp
- I2C – comm. With sensors
- USB – debugging,
- GPIO - for buzzer
- Timers – for delays, debugging
- SysTick - blink
- SPI – display
- Interrupts – waiting for UART message
- WiFi – esp communication

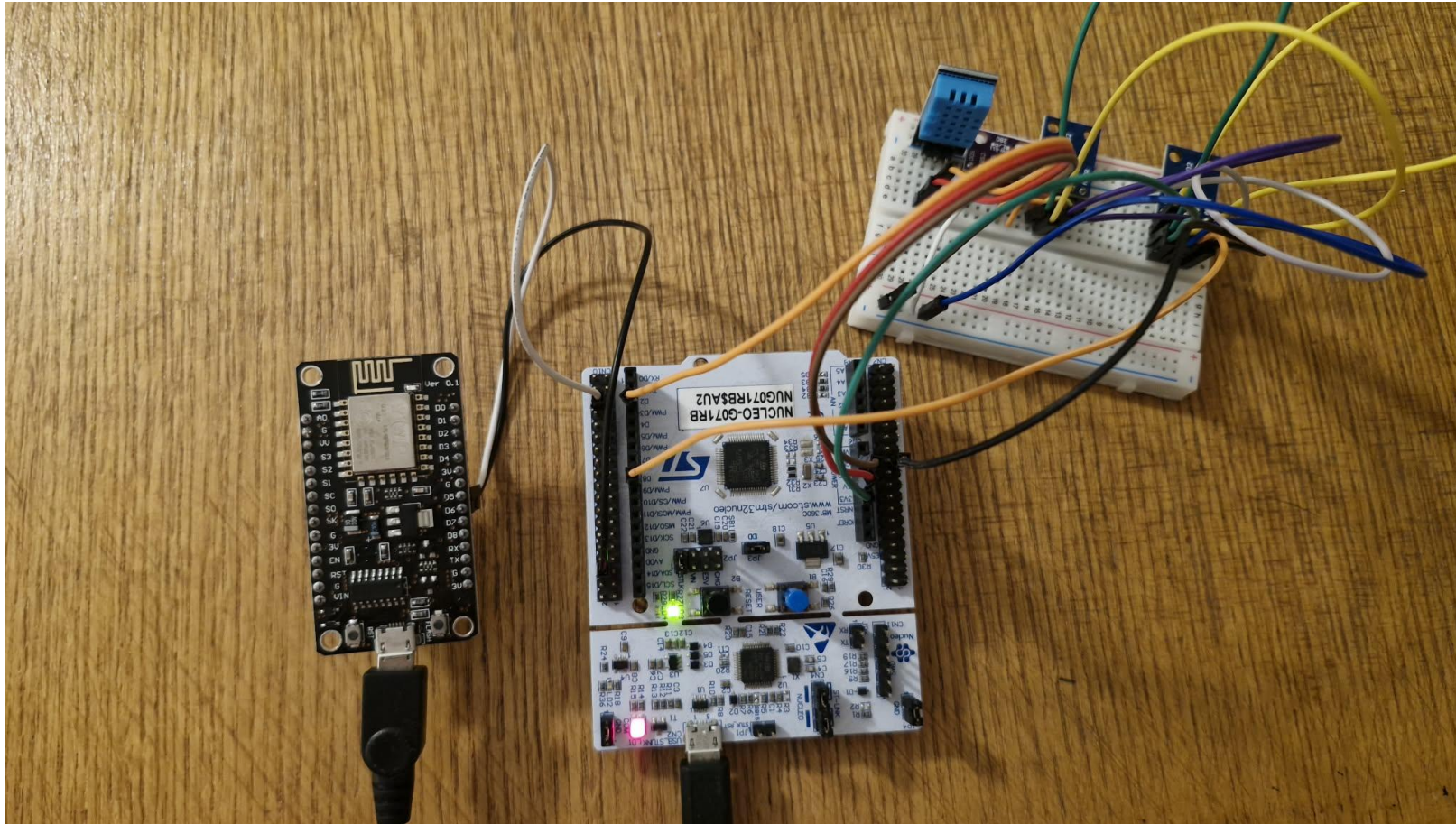
A description of the final result of the project

What has been achieved:

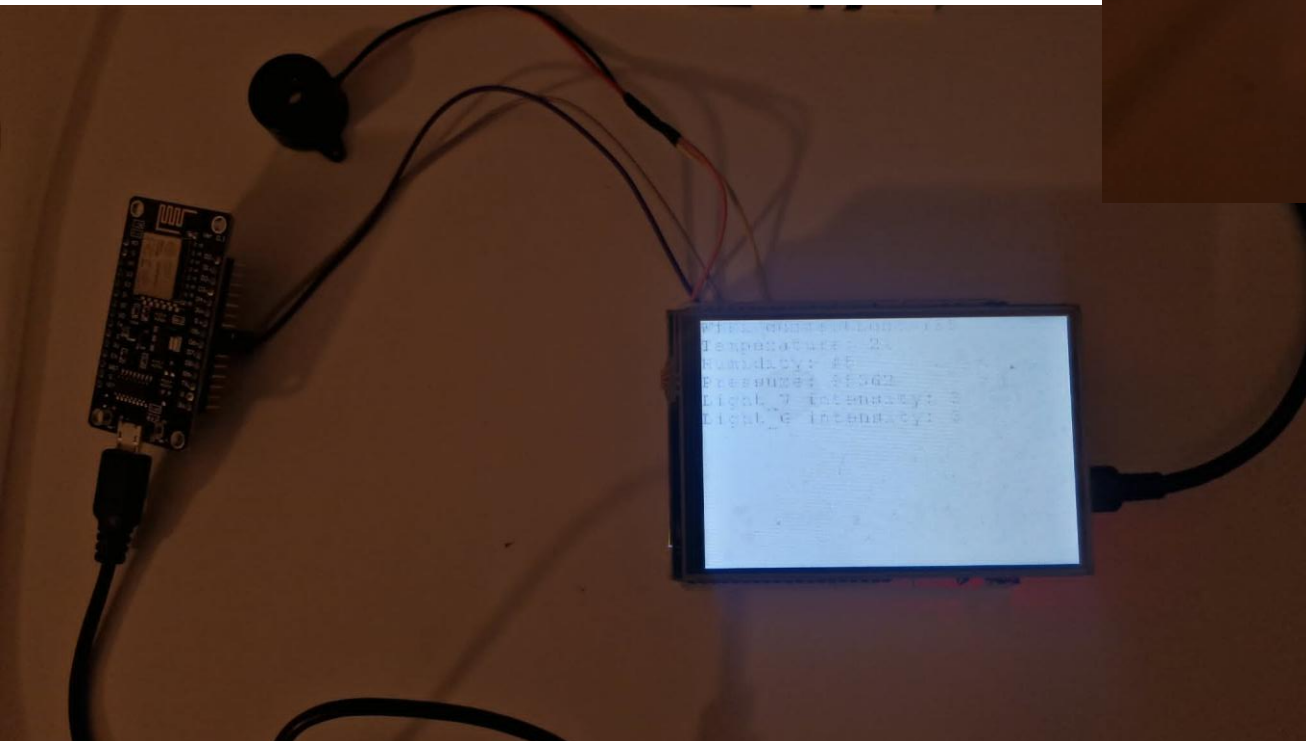
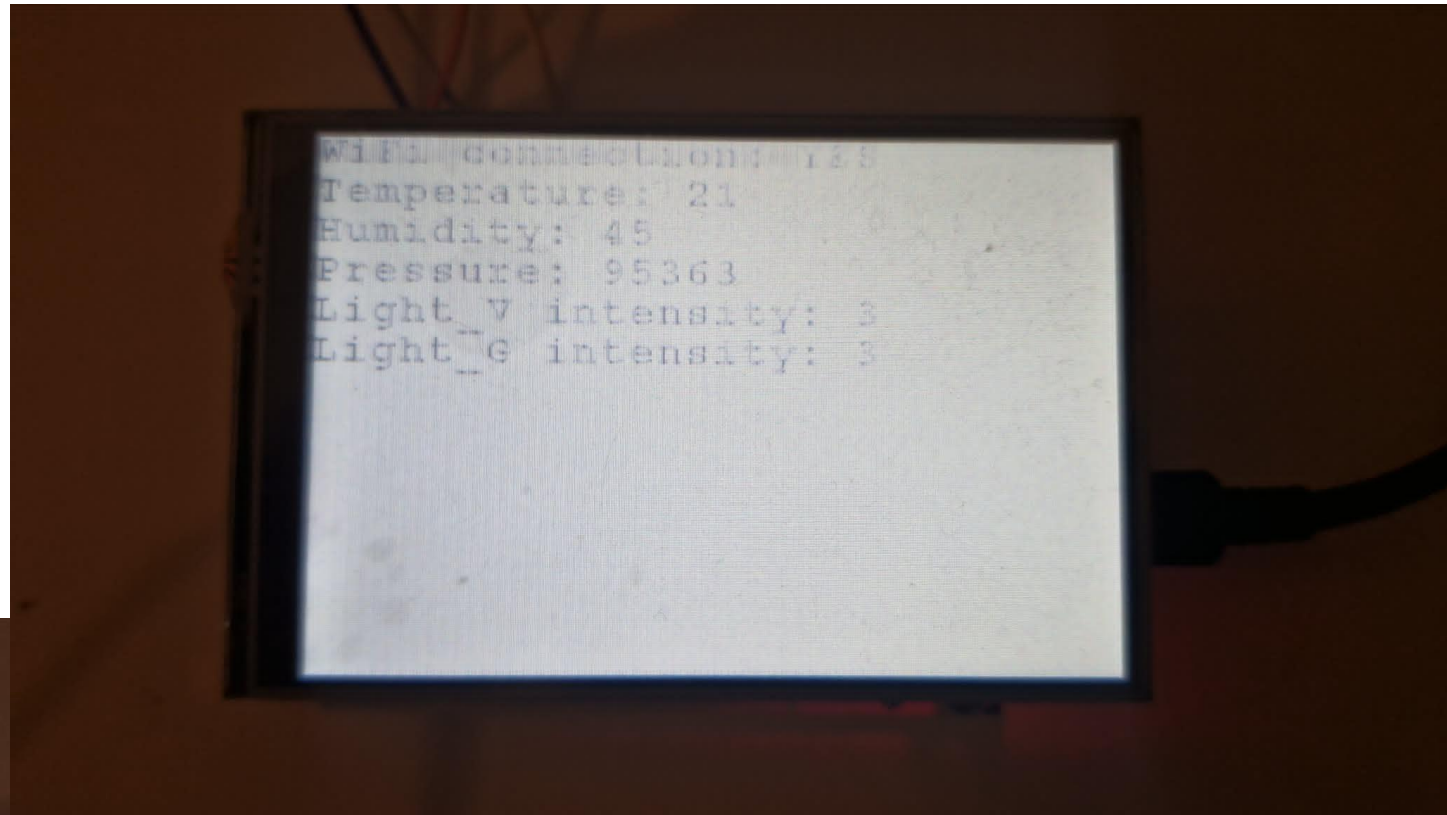
1. A simple program was created to display data collected from sensors
2. Sensor readings were implemented on the NUCLEO-G071RB
3. A buzzer was implemented based on humidity levels
4. The WiFi module was implemented
5. The project was successfully completed

Issue detected: At random intervals, the temperature and humidity readings show zero. The cause of this issue has not been determined. The problem disappears after resetting the microcontroller

Sensor circuit



Display unit



Comparison of the assumptions with the results obtained

1. Create simple program which displays something on display on stm32-L4 ✓
2. Implement sensors on stm32-G017 in order:
 - a) DHT11 ✓
 - b) + BMP280 ✓
 - c) + BH1750 ✓
3. Implement buzzer with conditions ✓
4. Implement WiFi module ✓
5. Make visualisation and send info through wifi ✓

Design must include:

1. Parallel processing ✓
2. Schematic in KiCad ✓ (github)
3. No user inputs ✓

repo

https://github.com/Equerm8/stm_wifi_project